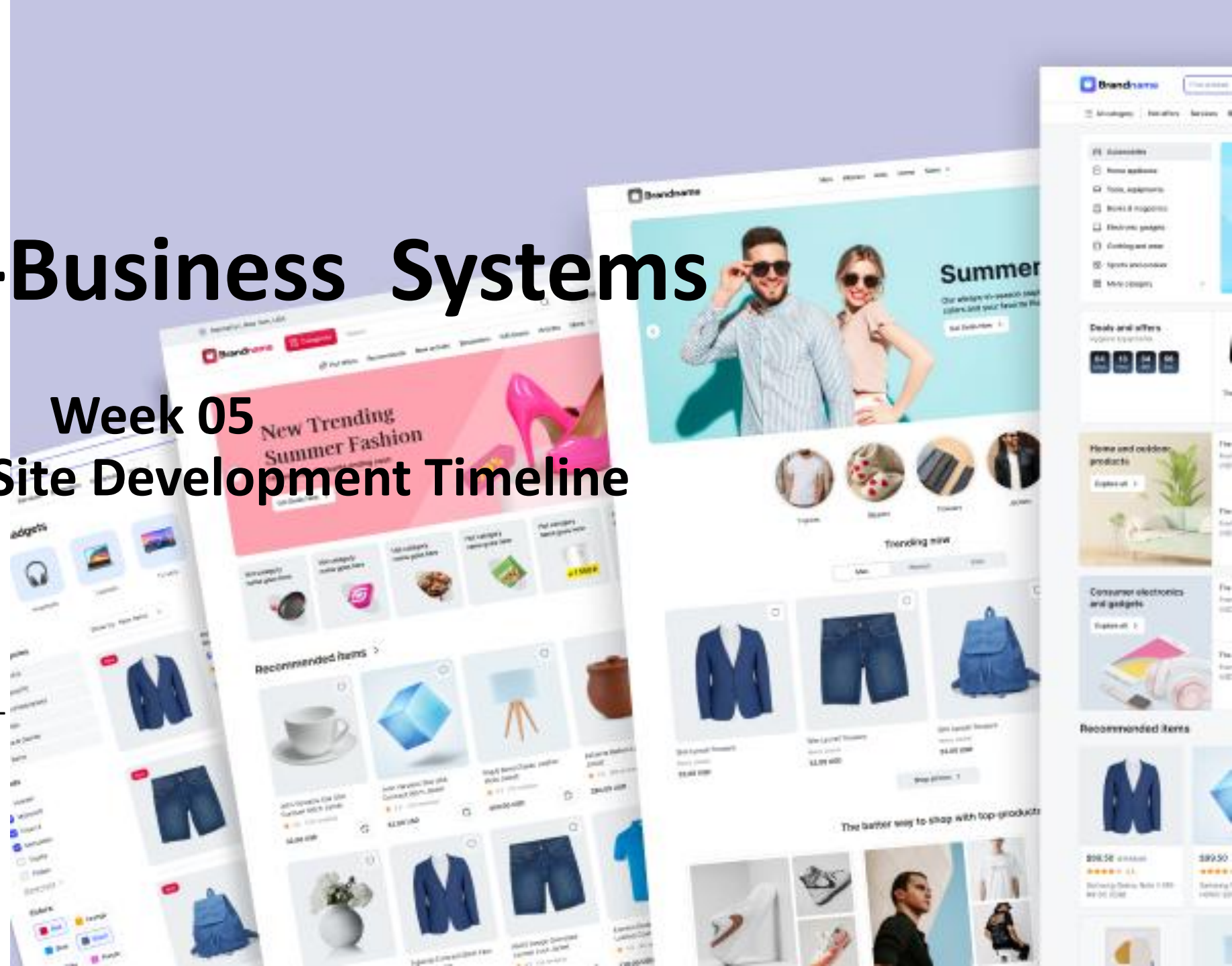


ICT2142 E-Business Systems

Week 05

E-Commerce Site Development Timeline

Phase 2 Core Development -
Frontend & Backend



Week 5: Shopping Cart Functionality



1. Week 5 Overview & Objectives
2. Step 1 - The "Add to Cart" Logic
3. Step 2 - "Remove Item" & Quantity Updates
4. Step 3 - Session Storage & State Persistence
5. Step 4 - Project Diary or Documentation

Week 5 Overview & Objectives

- **Core Goal:** Move from static product pages to an interactive store by building a working shopping cart.
- **Key Concept:** Students must choose where to temporarily store cart data:
 - Option A: Frontend State (e.g., React `useState`, Context API).
 - Option B: Browser Storage (`localStorage` or `sessionStorage` for plain JavaScript apps).

Step 1 - The "Add to Cart" Logic

What students must do:

- 1. Capture the Event:** Listen for a click event on the "Add to Cart" button from the Product Catalog or Product Detail page.
- 2. Pass Product Data:** Extract unique identifiers, such as **ProductID**, **Title**, **Price**, and **Image**.

Step 1 - The "Add to Cart" Logic (cont.)

3. Check Existing Items:

- Write a condition: Is the item already in the cart array?
- If Yes, increase its quantity by 1 instead of adding a duplicate row.
- If No, add it to the cart array with a quantity of 1.

4. Update Badge Counter: Dynamically update the cart icon number in the navbar so students get visual feedback.

Step 2 - "Remove Item" & Quantity Updates

What students must do:

1. **View Cart Page/Drawer:** Create a dedicated view where users can see all selected items listed out.
2. **Remove Item Feature:**
 - Attach a "Remove" or "Delete" button to each cart item.
 - Use an array method (like JavaScript `filter()`) to remove that specific product ID from the cart state.

Step 2 - "Remove Item" & Quantity Updates (cont.)

3. Quantity Up/Down Controls:

- Add + and - buttons next to each product.
- Write logic to increment or decrement the quantity.
- **Edge Case Rule:** If quantity drops below 1, either prompt the user or automatically remove the item from the cart.

4. Live Total Calculation: Automatically compute and display the subtotal and total cost based on Price x Quantity for all items in the cart.

Step 3 - Session Storage & State Persistence

- **The Problem:** If a user refreshes the webpage, standard JavaScript variables reset, and the cart empties!
- **The Solution:**
 - Introduce students to `localStorage` or `sessionStorage`.
 - Every time the cart changes (add, remove, update), convert the cart array into a JSON string and save it:
 - `localStorage.setItem('cart', JSON.stringify(cartArray));`
 - When the page loads, retrieve it:
 - `JSON.parse(localStorage.getItem('cart')) || [];`
- **Why this matters:** It gives students a realistic e-commerce experience where cart items persist through page reloads.

Step 4 - Project Diary & Documentation

What students must log in their documentation:

1. **State Architecture Notes:** Briefly write down how data flows from the product button into the cart state.
2. **Code Snippets / Logic Used:** Keep a record of the functions used for filtering out deleted items or handling local storage.
3. **Bug Log:** Document any state-sync issues faced (e.g., “Cart badge didn’t update instantly until manual refresh”) and how they resolved it.
4. **Git Commit:** Push the working cart implementation to GitHub with a clear commit message (e.g., “Implemented add-to-cart and localStorage persistence”).

Report Submission Requirement for Week 05

At the end of the week 03 practical session

Students should submit below sections as a word document as a part of the report with the given link in the LMS.

The report should include:

- **Answers for sections included under step 4.**

